

1/1/2022

NGÂN HÀNG LÝ THUYẾT NMCNPM PTIT

[6/2022]

HUY INIT
VŨ THANH TÚ
NGUYỄN VĂN TUẤN
PHẠM ANH TUẤN
CHU MINH HOÀNG
NGUYỄN HOÀNG TUẤN ANH
NGUYỄN VŨ
LÊ MẠNH DƯƠNG
TRẦN NGUYỄN ĐỨC ANH
TẠ ĐÌNH DUY
TRẦN KHÁNH HƯNG
NGÂN HÀNG CỦA 1 SỐ ANH CHỊ D11

HUY INIT
<https://www.facebook.com/huyinit13>

Contents

1. Thế nào là corrective maintenance?	5
2. Thế nào là adaptive maintenance?	5
3. Thế nào là perfective maintenance?	6
4. Thế nào là refactoring?	6
5. Thế nào là "from scratch"?	6
6. Thế nào là moving target problem?	6
7. Thế nào là regression fault?	6
8. Thế nào là một episode?	6
9. Thế nào là một iteration?	7
10. Thế nào là một increasement?	7
11. Thế nào là một artifact?	7
12. Thế nào là portability?	7
13. Thế nào là reuseability?	7
14. Thế nào là milestone?	7
15. Thế nào là một story?	8
16. Thế nào là refactoring?	8
17. Thế nào là concept exploration?	8
18. Thế nào là business model?	8
19. Thế nào là traceability?	8
20. Thế nào là egoless programming?	9
21. Thế nào là PM?	9
22. Thế nào là technical leader?	9
23. Thế nào là programming secretary?	9
24. Thế nào là backup programmer?	9
25. Thế nào là supper programmer?	9
26. Thế nào là một bản thiết kế còn ommision?	10
27. Thế nào là một bản thiết kế còn contradiction?	10
28. Thế nào là một phần mềm COTS?	10
29. Thế nào là SPMP?	10
30. Thế nào là alpha release?	10
31. Thế nào là beta release?	10
32. Thế nào là process?	11

33. Thế nào là workflow?	11
34. Luật Miller trong CNPM nói gì?.....	11
35. Luật Brooks trong CNPM nói gì?.....	11
36. Luật Dijkstra trong CNPM nói gì?.....	12
37. Verification và Validation (V&V) là gì?	12
38. Thế nào là inspection?	12
39. Thế nào walkthrough?.....	12
40. Thế nào là một moderator trong nhóm inspection?	12
41. Thế nào là một recorder trong nhóm inspection?.....	12
42. Mô hình CMM là gì?	13
43. Thế nào là test performance?	13
44. Thế nào là test robustness?.....	13
45. Thế nào là coin of uncertainty?.....	13
46. Thế nào là nominal effort?.....	14
47. Thế nào là phần mềm organic?	14
48. Thế nào là phần mềm embeded?.....	14
49. Thế nào là phần mềm semi-detached?	15
50. Thế nào là TCF?.....	15
51. Thế nào là UFP?.....	15
52. Thế nào là flow trong FFT?	15
53. Thế nào là process trong FFP?.....	15
54. Tại sao không có pha kiểm thử?	15
55. Tại sao không có pha làm tài liệu?.....	16
56. Tại sao không có pha lập kế hoạch?	16
57. Nếu không áp dụng các mô hình vòng đời phần mềm thì có phát triển được phần mềm không? Tại sao?	16
58. Tại sao người ta phải dùng nhiều mô hình vòng đời khác nhau để phát triển phần mềm?	16
59. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu thác nước?.....	17
60. Mô hình vòng đời phần mềm kiểu thác nước thì phù hợp với những dự án có đặc điểm gì?.....	17
61. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu bản mẫu nhanh?	18
62. Mô hình vòng đời phần mềm kiểu bản mẫu nhanh thì phù hợp với những dự án có đặc điểm gì?	18
63. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu lặp và tăng trưởng?	19
64. Mô hình vòng đời phần mềm kiểu lặp và tăng trưởng thì phù hợp với những dự án có đặc điểm gì?.....	19

65. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu xoắn ốc?	20
66. Mô hình vòng đời phần mềm kiểu xoắn ốc thì phù hợp với những dự án có đặc điểm gì?	20
67. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu tiến trình linh hoạt?	20
68. Mô hình vòng đời phần mềm kiểu tiến trình linh hoạt thì phù hợp với những dự án có đặc điểm gì?	21
69. Trong mô hình tiến trình liên hoạt, luôn có đại diện của khách hàng trong nhóm phát triển thì có ưu điểm gì?	22
70. Nêu ưu điểm, nhược điểm của mô hình nhóm code bình đẳng ((Democratic Team)?	22
71. Mô hình nhóm code bình đẳng thì phù hợp với những dự án có đặc điểm gì?	22
72. Nêu ưu điểm, nhược điểm của mô hình nhóm code có chef?	22
73. Mô hình nhóm code có chef thì phù hợp với những dự án có đặc điểm gì?	23
74. Nêu ưu điểm, nhược điểm của kỹ thuật pair programming?	23
75. Kỹ thuật pair programming thì phù hợp với những dự án có đặc điểm gì?	23
76. Nêu ưu điểm, nhược điểm của kỹ thuật time boxing?	23
77. Nêu ưu điểm, nhược điểm của kỹ thuật stand up meeting?	24
78. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng LOC?	24
79. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng FFP?	24
80. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng Function Point?	25
81. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng COCOMO?	25
82. Tại sao trong mô hình tiến trình linh hoạt, không cần có pha đặc tả?	25
83. Tại sao trong nhóm walkthrough và inspection, luôn phải có đại diện của workflow tiếp theo?	26
84. Nếu nhóm SQA phát hiện ra ít lỗi, thì có thể kết luận nhóm code giỏi hay nhóm SQA kém? Tại sao?	26
85. Tại sao nói inspection và walkthrough là hướng tài liệu, mà không phải hướng vào người tham gia?	26
86. Quality assurance thì khác gì với testing?	26
87. Tại sao nói function point chịu ảnh hưởng chủ quan của các chuyên gia?	27
88. COCOMO tính đến nhiều tiêu chí hơn hay là function point? Giải thích?	27
89. SW development multiplier của COCOMO thì khác gì TCF của function point?	27
90. TCF của function point thì khác gì hằng số b của FFP?	27
91. Tại sao nguyên lý Dijkstra lại đúng?	28
92. Tại sao luật Brook lại đúng?	28
93. Người ta áp dụng luật Miller trong CNPM như thế nào?	28
94. Phát triển phần mềm thì khác gì sản xuất phần mềm?	28
95. Test trường hợp sai kiểu dữ liệu đầu vào thì thuộc thể loại test gì?	28

Câu hỏi khác liên quan:.....	29
1. Phần mềm là gì? Nêu đặc trưng của nó. Có những loại ngôn ngữ nào để phát triển phần mềm?.....	29
2. Phân loại phần mềm và nội dung cơ bản mỗi loại.	29
3. Định nghĩa kỹ nghệ phần mềm? Những yếu tố chủ chốt trong kỹ nghệ phần mềm là gì?	29
4. Chất lượng phần mềm là gì? Các tiêu chí của chất lượng phần mềm.	29
5. Có các dạng bảo trì nào? Nêu và phân biệt.....	29
6. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu ổn định và đồng bộ hóa?.....	30
7. Mô hình vòng đời phần mềm kiểu ổn định và đồng bộ hóa thì phù hợp với những dự án có đặc điểm gì?.....	30
8. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu mã nguồn mở?	30
9. Mô hình vòng đời phần mềm kiểu mã nguồn mở thì phù hợp với những dự án có đặc điểm gì?	30
10. Tại sao trong mô hình tiến trình linh hoạt, không cần có pha đặc tả?.....	31
11. Nêu ưu, nhược điểm của phương pháp ước lượng phần mềm bằng COCOMO ?	31
12. Ý nghĩa của scenario và ngoại lệ ?.....	31
13. Ý nghĩa của sơ đồ tuần tự ?.....	31
14. Ý nghĩa của sơ đồ lớp ?.....	31
15. Thế nào là lớp giao diện ? Lớp này tương tác với các lớp nào ?	31
16. Thế nào là lớp điều khiển, lớp này tương tác với những lớp nào?.....	32
17. Thế nào là lớp thực thể, lớp này tương tác với những lớp nào?	32
18. Scenario và sơ đồ tuần tự có liên hệ gì đến nhau?	32
19. Scenario và sơ đồ lớp có quan hệ gì đến nhau?	32
20. Việc trích lớp và xây dựng các lớp là của pha thiết kế, tại sao người ta làm nó ngay trong pha phân tích?.....	32
21. Kỹ thuật trích danh từ dùng để trích các lớp nào? Có thể dùng để trích lớp biên và lớp điều khiển được không?.....	32
22. Trình bày kỹ thuật trích lớp điều khiển? Số lượng lớp điều khiển nhiều hay ít thì tốt?	33
23. Ý nghĩa của sơ đồ trạng thái hữu hạn? Nó biểu diễn trạng thái của cả hệ thống, của lớp hay của phương thức?	33
24. Ý nghĩa thẻ CRC, dùng thẻ CRC với lớp biên và lớp điều khiển được không? Có cần thiết không?.....	33
25. Thiết kế kiến trúc cần sơ đồ quan hệ các lớp (Communication Diagram).....	33
26. Thiết kế chi tiết cần Sơ đồ cần dùng.....	33
27. Làm thế nào để trích các lớp? Cần dùng các sơ đồ nào của UML?.....	33
28. Sơ đồ lớp và sơ đồ cộng tác có gì khác nhau?	33
29. Mỗi trạng thái của sơ đồ trạng thái ứng với một lớp hay một phương thức? Tại sao?	33

30. Trình bày nguyên lý A của phần gán phương thức cho lớp, nguyên lý này	34
31. Trình bày nguyên lý B của phần gán phương thức cho lớp, nguyên lý này thường dùng cho lớp loại nào?	34
32. Trình bày nguyên lý C của phần gán phương thức cho lớp, nguyên lý này thường dùng cho lớp loại nào?	34
33. inspection và walkthrough?	34
34. Người ta nói « nhóm SQA tạo ra chất lượng cho phần mềm » đúng hay sai? Tại sao?.....	35
35. Trình bày phương pháp kiểm thử hộp trắng: cơ sở phương pháp; các yêu cầu cần kiểm tra, các kỹ thuật được sử dụng.....	35
36. Trình bày phương pháp kiểm thử hộp đen: cơ sở phương pháp; các yêu cầu cần kiểm tra, các kỹ thuật được sử dụng.....	35
37. Kiểm thử đơn vị đối tượng là gì? Ai thực hiện. Các phương pháp và kỹ thuật nào được sử dụng? Kiểm tra những loại lỗi nào?.....	36
38. Giải thích khái niệm stub và driver? Chúng được sử dụng ở đâu và vì sao?	36
39. Kiểm thử hệ thống nhằm kiểm tra cái gì? Ai thực hiện? Các phương pháp?	36
40. Kiểm thử chấp nhận là gì? Trong đó có những kiểm thử nào được thực hiện? Phân biệt	37
41. Mô hình CMM là gì? Có những mức tăng trưởng nào trong mô hình CMM? Nội dung của mỗi mức?.....	37
42. Làm thế nào để một tổ chức đạt được các mức tăng trưởng trong CMM?	38

1. Thế nào là corrective maintenance?

Trả lời :

-Corrective maintenance(bảo trì khắc phục): là các hoạt động khắc phục sự cố khi một thiết bị hoặc máy móc không thực hiện đúng chức năng của nó, khắc phục khi một bộ phận của máy móc bị hư hỏng, khi vấn đề được phát hiện. Các lỗi này có thể do lỗi thiết kế, lỗi logic hoặc lỗi coding sản phẩm. Bên cạnh đó, các lỗi cũng có thể do quá trình xử lý dữ liệu, hoặc hoạt động của hệ thống.

-Lợi ích của việc corrective maintenance:

- +Chi phí bảo trì thấp
- +Quy trình đơn giản
- +Ít khi cần đến việc lên kế hoạch

2. Thế nào là adaptive maintenance?

Trả lời:

-Adaptive maintenance(bảo trì thích nghi): là tiến hoá hệ thống, chỉnh sửa phần mềm nhằm phù hợp với môi trường đã thay đổi của sản phẩm.

-Adaptive maintenance được sử dụng với mục đích:

- +Cạnh tranh
- +Nhu cầu nội bộ

3. Thế nào là perfective maintenance?

Trả lời:

-Perfective maintenance(bảo trì hoàn chỉnh): là bảo trì phần mềm sau khi giao hàng được thực hiện để cải thiện chức năng và hiệu suất của chương trình hoặc tăng khả năng bảo trì. Việc Hành động này giúp phát hiện và sửa chữa các lỗi tiềm ẩn trong sản phẩm phần mềm trước khi chúng được biểu hiện dưới dạng lỗi .

4. Thế nào là refactoring?

Trả lời :

-Refactoring là định nghĩa ngắn về cải tiến và làm tốt hơn chất lượng của mã nguồn trong 1 ứng dụng. Nó không làm thay đổi các chức năng chính, chức năng chung của ứng dụng, nhưng nó làm cho ứng dụng dễ bảo trì hơn, dễ phát triển hơn trong tương lai.

5. Thế nào là "from scratch"?

Trả lời:

-From scratch là xây dựng phần mềm từ đầu (làm mới hoàn toàn) mà không dựa trên 1 cơ sở hay 1 phần mềm có sẵn nào từ trước .

6. Thế nào là moving target problem?

Trả lời :

-Moving target problem là một sự thay đổi yêu cầu khi sản phẩm phần mềm đang được phát triển , được sử dụng khi người dùng (client) thay đổi yêu cầu về chức năng phần mềm sau khi yêu cầu chức năng cũ đã được thêm vào phần mềm.

7. Thế nào là regression fault?

Trả lời:

-Regression fault (lỗi hồi quy) là một loại lỗi phần mềm trong đó một tính năng đã hoạt động trước khi ngừng hoạt động. Lỗi này có thể xảy ra sau khi các thay đổi được áp dụng cho sourcecode của phần mềm, bao gồm cả việc bổ sung các tính năng mới và sửa lỗi.

8. Thế nào là một episode?

Trả lời:

-Episode là lần lặp lại các pha.

Ví dụ minh họa: khi thực hiện xong việc phân tích tới việc cài đặt chương trình, ta phát hiện ra bản thiết kế không chính xác hoặc có lỗi thì ta phải thiết kế lại và cài đặt lại thì đó chính => là 1 pha lặp lại (episode). Và phần mềm sẽ được làm xong và giao tới khách hàng sau nhiều pha lặp lại.

9. Thế nào là một iteration?

Trả lời:

-Iteration (Sự lặp đi lặp lại) là một cách tiếp cận phát triển phần mềm, tách quá trình phát triển một ứng dụng lớn thành các phần nhỏ hơn. Mỗi phần, được gọi là “lặp lại”, đại diện cho toàn bộ quá trình phát triển và chứa các bước lập kế hoạch, thiết kế, phát triển và thử nghiệm.

10. Thế nào là một increasement?

Trả lời:

-Increment (sự gia tăng) là một phương pháp phát triển phần mềm trong đó sản phẩm được thiết kế, triển khai và thử nghiệm từng bước (mỗi lần thêm một ít) cho đến khi sản phẩm hoàn thành. Nó liên quan đến cả phát triển và bảo trì.

11. Thế nào là một artifact?

Trả lời:

- Artifact (đồ tạo tác) là một trong nhiều loại sản phẩm hữu hình được tạo ra trong quá trình phát triển phần mềm. Một số artifact (ví dụ như: use case, sơ đồ lớp, các mô hình UML khác, yêu cầu và các tài liệu thiết kế) sẽ giúp mô tả chức năng, kiến trúc và thiết kế cho phần mềm. Mỗi artifact khác nhau thì liên quan với quy trình phát triển riêng của nó - ví dụ như kế hoạch dự án, các business case và đánh giá rủi ro.

12. Thế nào là portability?

Trả lời:

- Portability (tính khả chuyển) của phần mềm là mức độ dễ dàng trong việc chỉnh sửa toàn bộ phần mềm để có thể chạy được trên một hệ thống, môi trường, phần cứng khác mà không phải lập trình lại từ đầu.

13. Thế nào là reuseability?

Trả lời:

- Reuseability (tính sử dụng lại) của phần mềm là quá trình phát triển phần mềm từ các thành phần phần mềm hiện có, thay vì phát triển toàn bộ phần mềm từ đầu.

14. Thế nào là milestone?

Trả lời:

-Milestone (cột mốc quan trọng của dự án) là một công cụ quản lý được sử dụng để xác định một mốc trong lịch trình dự án. Những mốc này có thể ghi nhận sự bắt đầu và kết thúc của một dự án, và đánh dấu sự hoàn thành của một giai đoạn công việc chính.

15. Thế nào là một story?

Trả lời:

- User story (câu chuyện người dùng) là một mô tả không chính thức bằng ngôn ngữ tự nhiên về một hoặc nhiều tính năng của hệ thống phần mềm. User story mô tả kiểu người dùng, họ muốn gì và tại sao. User story giúp tạo ra một mô tả đơn giản về một yêu cầu.

16. Thế nào là refactoring?

Trả lời :

-Refactoring là định nghĩa ngắn về cải tiến và làm tốt hơn chất lượng của mã nguồn trong 1 ứng dụng. Nó không làm thay đổi các chức năng chính, chức năng chung của ứng dụng, nhưng nó làm cho ứng dụng dễ bảo trì hơn, dễ phát triển hơn trong tương lai.

17. Thế nào là concept exploration?

Trả lời:

-Concept exploration (Tìm hiểu khái niệm hay còn có tên gọi khác là requirements-pha yêu cầu) là pha đầu tiên trong quá trình xây dựng phần mềm, khi đó người phát triển và khách hàng ngồi lại với nhau. Khách hàng nêu ra những yêu cầu mà phần mềm cần có còn những người phát triển ghi chép lại. Khám phá khái niệm xác định dự án có triển vọng và tính khả thi để phát triển.

18. Thế nào là business model?

Trả lời:

-Business model (mô hình kinh doanh): là mô hình diễn tả kế hoạch nhằm phân phối sản phẩm của bạn tới tay khách hàng. Đó là lời giải thích tổng quát về những giá trị mà bạn cung cấp cho khách hàng của mình với chi phí phù hợp để doanh nghiệp của bạn có thể kiếm tiền.

-Những cách để phân phối sản phẩm phần mềm cho khách hàng:

+Tạo ra sản phẩm cài đặt toàn bộ ngay trên máy tính khách hàng

+Tạo ra sản phẩm trên cloud dựa vào việc lưu trữ máy chủ trên đám mây (còn được gọi là SaaS - Software as a service)

+Kết hợp cả hai phương pháp trên

19. Thế nào là traceability?

Trả lời:

-Traceability: là khả năng theo dõi và truy vết công việc trong suốt vòng đời của ứng dụng. Nó thường được sử dụng để theo dõi quá trình phát triển phần mềm

và những thứ đã xảy ra trong từng giai đoạn, tất cả mọi thứ đều phải được kiểm thử.

20. Thế nào là egoless programming?

Trả lời:

-Egoless programming(lập trình tập thể) là một phong cách lập trình máy tính trong đó các yếu tố cá nhân được giảm thiểu để chất lượng có thể được cải thiện.

21. Thế nào là PM?

Trả lời:

-PM (hay Product Manager) là người chịu trách nhiệm sản phẩm , giám sát và định hướng việc phát triển một sản phẩm (product) nào đó, ví dụ như là một tính năng mới cho một app. Những quyết định của Product Manager thường được thực hiện thông qua việc phân tích dữ liệu để tìm ra vấn đề của sản phẩm, từ đó tối ưu hóa trải nghiệm và cải thiện phản ứng của người dùng đối với sản phẩm.

22. Thế nào là technical leader?

Trả lời:

-Technical leader là người đảm nhiệm việc quản lý và phân phối các dự án kỹ thuật, thường là trong bối cảnh của phần mềm máy tính. Họ quản lý các nhóm Developer và làm việc chặt chẽ với các nhà quản lý cấp cao để đảm bảo các dự án được giao đúng thời hạn và trong phạm vi ngân sách.

23. Thế nào là programming secretary?

Trả lời:

-Programming secretary (Thư ký lập trình) là người phụ trách việc hỗ trợ ghi chép, sắp xếp và thông báo các công việc, đề xuất, tin tức cho giám đốc doanh nghiệp.

24. Thế nào là backup programmer?

Trả lời:

-Backup programmer (lập trình viên dự bị): là người có năng lực nhưng không được tham gia chính thức trong dự án, người này sẽ ở ngoài và sẵn sàng thay thế khi một lập trình viên ko thể tiếp tục với dự án vì một lý do nào đó.

25. Thế nào là supper programmer?

Trả lời:

-Super programmer là người khiêm tốn và sẵn sàng chịu trách nhiệm về sai lầm cũng như học hỏi từ sai lầm, viết code có cấu trúc, có thể đọc được, thiết kế code vững chắc có thể được gỡ lỗi dễ dàng, Có thể bắt kịp với việc học hỏi các công nghệ mới, Thích giải quyết vấn đề.

26. Thế nào là một bản thiết kế còn omission?

Trả lời:

-Một bản thiết kế còn omission (thiết sót) là bản thiết kế còn nhiều sự mập mờ, thiếu rõ ràng.

27. Thế nào là một bản thiết kế còn contradiction?

Trả lời:

-Một bản thiết kế còn contradiction (sự mâu thuẫn) bản thiết kế tồn tại mâu thuẫn dẫn đến phần mềm sau này có thể không hoạt động được ở chức năng đó.

28. Thế nào là một phần mềm COTS?

Trả lời:

-Commercial Off-the-Shelf Software (COTS) là một phần mềm được sản xuất và bán với mục đích thương mại trong một cửa hàng bán lẻ và trực tuyến, sẵn sàng sử dụng mà người dùng không có bất kỳ hình thức sửa đổi nào và mọi người đều có thể truy cập được=> Đây là phần mềm dùng cho nhiều người

29. Thế nào là SPMP?

Trả lời:

-SPMP (Software Project Management Plan): Kế hoạch quản lý dự án phần mềm là tài liệu kiểm soát cho việc quản lý một dự án phần mềm. Nó bao gồm: ngân sách, yêu cầu nhân sự và lịch trình chi tiết. Thời điểm sớm nhất để ta lên SPMP là khi tài liệu đặc tả đã được khách hàng chấp thuận, nghĩa là vào cuối giai đoạn phân tích. Cho đến lúc đó, kế hoạch phải được chuẩn bị sơ bộ.

30. Thế nào là alpha release?

Trả lời:

-Alpha release là phiên bản được kiểm thử hoạt động chức năng thực tế hoặc giả lập do một số ít người dùng/khách hàng được chỉ định hoặc một nhóm test thực hiện tại nơi sản xuất phần mềm. Alpha release được thực hiện ngay sau kiểm thử hệ thống (Product Testing). Alpha release thường áp dụng cho các sản phẩm COTS (MS Office, Windows,...) là một hình thức kiểm thử chấp nhận nội bộ, trước khi phần mềm được tiến hành kiểm thử beta.

31. Thế nào là beta release?

Trả lời:

-Beta release: là bản phát hành cho một bộ phận khách hàng thử nghiệm bản alpha release đã sửa chữa. Beta release gần như phiên bản cuối cùng (final version).

32. Thế nào là process?

Trả lời:

-Process(tiến trình): là cách thức sản xuất ra phần mềm. Bao gồm nhiều quy trình trong việc tạo ra 1 phần mềm, có thể có nhiều mô hình khác nhau trong 1 process, có thể sử dụng mô hình workflow để làm ra phần mềm.

-Process khác với workflow là

- + bao gồm nhiều quy trình trong việc tạo ra 1 phần mềm
- + Có thể có nhiều mô hình khác nhau trong 1 process
- + Process có thể sử dụng mô hình workflow để làm ra phần mềm

33. Thế nào là workflow?

Trả lời:

-Workflow (Luồng): là các định nghĩa của các qui trình đã được chuẩn hóa và khi mình viết các module cho từng công việc, workflow là 1 chuỗi công việc phải làm. Workflow là 1 cách thực hiện cụ thể của process.

-Một workflow bao gồm 5 công việc:

- +Requirements workflow
- +Analysis workflow
- +Design workflow
- +Implementation workflow
- +Test workflow

34. Luật Miller trong CNPM nói gì?

Trả lời:

-Định luật Miller : Tại mỗi thời điểm, người ta chỉ có thể tập trung vào tối đa khoảng 7 vấn đề

=> Áp dụng luật Miller trong CNPM → để xử lý các vấn đề lớn, sử dụng phương pháp làm mịn từng bước: ta phải tập trung xử lý các việc quan trọng trước, các việc ít quan trọng hơn xử lý sau → gọi là tiến trình tăng trưởng

35. Luật Brooks trong CNPM nói gì?

Trả lời:

-Đây là một định luật dựa trên kinh nghiệm thực tế : “Đưa thêm người vào 1 project đang chậm, sẽ chỉ khiến nó càng chậm hơn”. Ví dụ chứng minh : “Tập hợp 9 bà bầu lại cũng không thể đẻ được đứa trẻ ra đời sau 1 tháng.”

-Luận thuyết cơ bản của định luật này là:

- +Cần thời gian để quen với project
- +Công sức dành cho việc communication sẽ tăng

36. Luật Dijkstra trong CNPM nói gì?

Trả lời:

-Luật Dijkstra: Rất dễ để chứng minh một phần mềm có lỗi, nhưng không thể chứng minh được một phần mềm không có lỗi. Ví dụ chứng minh : code 1 sản phẩm ta chỉ ra được 1000000 test case nhưng ta cũng không thể chắc chắn được rằng test thứ 1000001 sẽ không có lỗi .

37. Verification và Validation (V&V) là gì?

Trả lời:

- Verification (xác minh, kiểm định): là quá trình đánh giá các sản phẩm làm việc trung gian của một vòng đời phát triển phần mềm để kiểm tra xem liệu rằng chúng ta có đi đúng hướng để tạo ra sản phẩm cuối cùng.

-Validation (xác nhận, thẩm định) :là quá trình đánh giá sản phẩm cuối cùng để kiểm tra xem phần mềm có đáp ứng được yêu cầu nghiệp vụ không? Hoạt động validation bao gồm smoke testing, functional testing, regression testing, systems testing etc...

38. Thế nào là inspection?

Trả lời:

-Inspection là sự kiểm tra hoặc xem xét các nhóm chính thức, một cách công khai, nghiêm ngặt, bài bản và chuyên sâu để xác định các vấn đề càng sớm càng tốt. Thường được dẫn dắt bởi người kiểm duyệt được đào tạo bài bản và có trình độ chuyên môn cao. Mục đích ở đây là chỉ ra và ghi chép lại các lỗi không phải để sửa lỗi.

39. Thế nào là walkthrough?

Trả lời:

-Walkthrough là cuộc họp không chính thức, thường không có người điều hành, và không có kế hoạch. Walkthrough có ưu điểm là tốn ít thời gian. Người viết code (tác giả) sẽ thể hiện sản phẩm mình đang làm, và đồng đội sẽ đưa ra các khiếm khuyết và các đề xuất. Tác giả sẽ tự ghi chép lại những đề xuất và cải thiện sản phẩm của mình.

40. Thế nào là một moderator trong nhóm inspection?

Trả lời:

-Moderator là người điều hành, chủ trì, chịu trách nhiệm chung và trong thời gian một ngày phải có báo cáo văn bản về việc kiểm tra để đảm bảo theo dõi tỉ mỉ.

41. Thế nào là một recorder trong nhóm inspection?

-recorder là thư ký chịu trách nhiệm ghi chép và phân loại các vấn đề gặp phải trong các cuộc họp. Họ sẽ hỗ trợ Moderator (người giám sát) trong việc chuẩn

bị báo cáo, ghi chép những vấn đề nào sẽ được chỉnh sửa, có bao nhiêu defect cần phải chỉnh sửa, có bao nhiêu ý kiến trong một defect,...

42. Mô hình CMM là gì?

Trả lời:

-CMM (Capability maturity model) là các chiến lược có thể sử dụng để cải thiện quy trình phần mềm mà không phụ thuộc vào bất cứ một mô hình vòng đời phần mềm (life-cycle model) nào cả.

-Mô hình CMM gồm 5 mức:

- +Initial (Mức khởi đầu)
- +repeatable (Mức có khả năng lặp lại)
- +defined (Mức có được định nghĩa)
- +managed (Mức có được quản lý)
- +optimize (Mức tối ưu hóa)

43. Thế nào là test performance?

Trả lời:

-Performance Testing (Kiểm tra hiệu năng (hiệu suất)) là một kỹ thuật kiểm tra phần mềm phi chức năng nhằm xác định mức độ ổn định, tốc độ, khả năng mở rộng và khả năng đáp ứng của ứng dụng được duy trì như thế nào trong một khối lượng công việc nhất định. Performance Testing thường được sử dụng để:

- +Đánh giá mức độ sẵn sàng của sản phẩm
- +Đánh giá dựa vào các tiêu chí hiệu suất
- +So sánh giữa các đặc tính hiệu suất của đa hệ thống hoặc cấu hình hệ thống
- +Tìm ra nguồn gốc của các vấn đề về hiệu suất
- +Hỗ trợ điều chỉnh hệ thống
- +Tìm các mức độ băng thông

44. Thế nào là test robustness?

Trả lời:

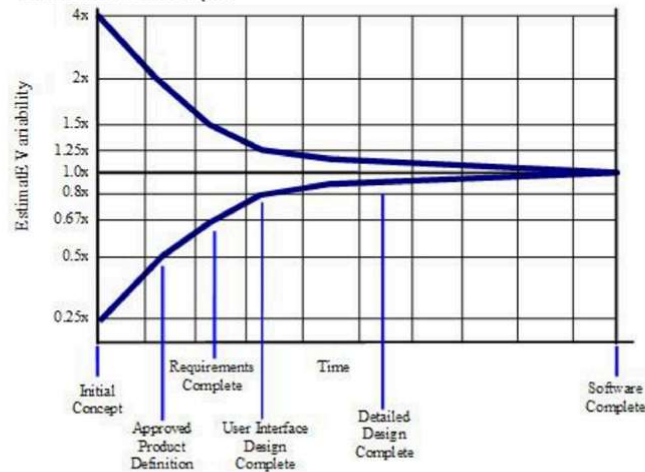
-Robustness testing (Kiểm tra độ mạnh) là một phương pháp kiểm tra đảm bảo chất lượng để kiểm tra tính mạnh mẽ của ứng dụng phần mềm. Ở đây việc kiểm tra được thực hiện bằng cách đưa ra các đầu vào không hợp lệ. Kiểm tra độ chắc chắn thường được thực hiện để kiểm tra khả năng xử lý đặc biệt.

45. Thế nào là coin of uncertainty?

Trả lời:

-Cone of uncertainty (Hình nón của sự không chắc chắn) :Là hình nón tạo bởi cận trên và cận dưới của chi phí ước tính trong từng pha. Trong giai đoạn đầu của một dự án, các chi tiết cụ thể về bản chất của phần mềm sẽ được xây dựng, chi tiết về các yêu cầu cụ thể, giải pháp, kế hoạch, nhân sự là không rõ ràng =>Với những pha càng sớm, hai cận này càng chênh nhau. Khi các chênh lệch này về sau được điều tra và xác định rõ hơn, sự biến động trong dự án giảm dần, và do đó độ biến động trong các ước tính của dự án *cũng* có thể giảm đi. Đó chính là hiện tượng “Hình nón của sự không chắc chắn”

-Hình ảnh minh họa :



46. Thế nào là nominal effort?

Trả lời:

-Nominal effort là nỗ lực danh nghĩa, ngụ ý rằng nếu bất kỳ ai cố gắng hoàn thành dự án trong thời gian ngắn hơn thời hạn ước tính, thì chi phí sẽ tăng lên đáng kể. Nhưng, nếu bất kỳ ai hoàn thành dự án trong một khoảng thời gian dài hơn ước tính, thì giá trị chi phí ước tính hầu như không giảm.

47. Thế nào là phần mềm organic?

Trả lời:

-Phần mềm organic là một phần mềm mà số lượng nhân lực cần thiết để triển khai là tương đối nhỏ, vấn đề thường đã được hiểu kỹ và đã từng được giải quyết trong quá khứ.

48. Thế nào là phần mềm embedded?

Trả lời:

-Phần mềm embedded là phần mềm được viết cho một mục đích cụ thể dựa vào một phần của phần cứng, phần mềm có quy mô phức tạp.

49. Thế nào là phần mềm semi-detached?

Trả lời:

-Phần mềm semi-detached là phần mềm có các đặc điểm quan trọng như quy mô nhóm, kinh nghiệm, kiến thức về môi trường lập trình khác nhau nằm giữa môi trường organic và embedded. Các dự án phần mềm loại Semi-detached là tương đối ít quen thuộc và khó phát triển hơn so với các dự án organic và đòi hỏi nhiều kinh nghiệm hơn, sự điều khiển tốt hơn và sáng tạo hơn.

50. Thế nào là TCF?

Trả lời:

-TCF (Technical complexity factor) : là hệ số được sử dụng để điều chỉnh kích thước dựa trên các cân nhắc kỹ thuật. Đây là thước đo tác dụng của 14 yếu tố kỹ thuật, chẳng hạn như high transaction rates, performance criteria và online update.

51. Thế nào là UFP?

Trả lời:

-UFP (Unadjusted function points) : là các điểm chức năng được chỉ định cho mỗi thành phần sau đó tổng hợp lại thành các điểm chức năng chưa được điều chỉnh, xác định bởi external input, external output, external enquiries, internal logical files, external interface files

52. Thế nào là flow trong FFT?

Trả lời:

-Flow trong FFT là một giao diện dữ liệu giữa sản phẩm và môi trường, chẳng hạn như một màn hình hoặc một báo cáo.

53. Thế nào là process trong FFP?

Trả lời:

-Process trong FFP là một thông số để tính kích thước sản phẩm bằng FFP, là một thao tác logic hoặc số học chức năng của dữ liệu. Ví dụ bao gồm sorting, validating, or updating

54. Tại sao không có pha kiểm thử?

Trả lời:

-Không có pha kiểm thử vì chương trình được kiểm tra tại cuối mỗi workflow và khi hoàn thành sản phẩm, kiểm thử phải được thực hiện xuyên suốt trong quá trình phát triển phần mềm chứ không phải làm 1 lần là xong

-Nếu không kiểm thử tính đúng đắn của phần mềm ở từng giai đoạn mà chỉ kiểm ở giai đoạn cuối và phát hiện ra lỗi, thì thường bàn giao sản phẩm không đúng hạn .

55. Tại sao không có pha làm tài liệu?

Trả lời:

-Không có pha làm tài liệu vì tài liệu được làm và thực hiện sửa đổi liên tục trong quá trình làm phần mềm. Tất cả những cái mình tạo ra trong lúc làm sản xuất đều được coi là tài liệu: tài liệu đặc tả, phân tích, thiết kế, coding, testing,... Vì tất cả đều cần làm tài liệu nên không có pha làm tài liệu riêng.

56. Tại sao không có pha lập kế hoạch?

Trả lời:

-Không có pha lập kế hoạch vì chúng ta không thể lập kế hoạch vào đầu dự án mà chúng ta chưa biết chính xác những gì mà chúng ta sẽ xây dựng. Chúng ta chỉ có thể lập kế hoạch sơ bộ cho pha yêu cầu và pha phân tích khi bắt đầu mỗi dự án. Kế hoạch quản lý dự án phần mềm chỉ dc đưa ra khi các chi tiết kỹ thuật mà khách hàng đưa ra đã được hoàn tất, vì thế kế hoạch phải được thực hiện xuyên suốt trong quá trình phát triển phần mềm
-Vì chỉ có bản kế hoạch tạm thời về quản lý dự án, mọi kế hoạch chỉ là ước lượng, quá trình có thể bị thay đổi do nhiều tác nhân khác nhau trong lúc thực hiện dự án.

57. Nếu không áp dụng các mô hình vòng đời phần mềm thì có phát triển được phần mềm không? Tại sao?

Trả lời:

-Nếu không áp dụng các mô hình vòng đời thì rất khó để phát triển phần mềm vì chưa biết là dự án đang phát triển ở giai đoạn nào và không biết nên làm gì tiếp theo. Do đó khó kiểm soát được phần mềm (nhiều lỗi tiềm ẩn, khả năng gắn kết các module kém), khả năng tái sử dụng module kém, khó phát triển phần mềm để đạt được yêu cầu với mỗi phần mềm riêng biệt.

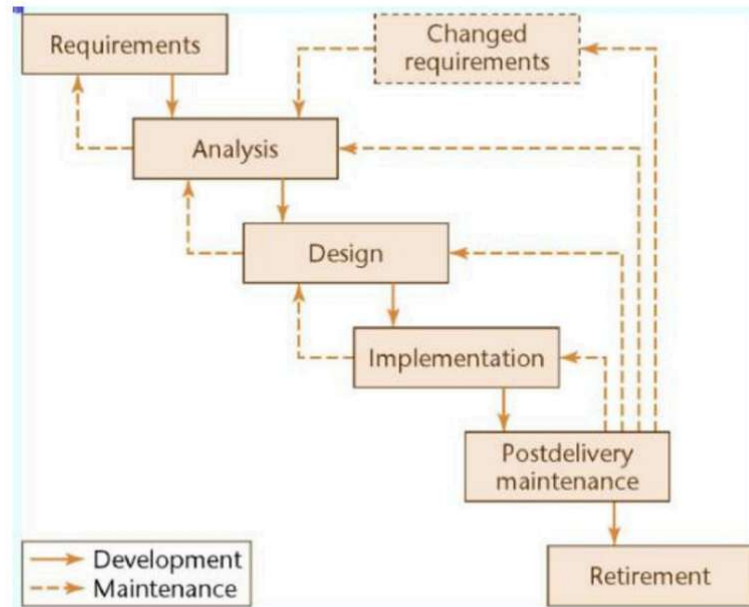
58. Tại sao người ta phải dùng nhiều mô hình vòng đời khác nhau để phát triển phần mềm?

Trả lời:

-Phải dùng nhiều mô hình vòng đời khác nhau vì các mô hình vòng đời có các ưu điểm, nhược điểm khác nhau phù hợp với những điều kiện phát triển phần mềm khác nhau. Quy mô của các dự án phát triển phần mềm khác nhau nên đòi hỏi các mô hình vòng đời khác nhau phù hợp với kinh phí phát triển dự án đó.

59. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu thác nước?

Trả lời:



-Ưu điểm:

+Dễ phân công công việc, phân bổ chi phí, giám sát và quản lý công việc.

+Xác nhận ở từng giai đoạn đảm bảo ít lỗi

+Kiến trúc chặt chẽ, rõ ràng

-Nhược điểm:

+Ít linh hoạt, phạm vi điều chỉnh bị hạn chế

+Khó quay lại khi giai đoạn nào đó kết thúc => ko phù hợp các dự án thay đổi nhiều yêu cầu.

+Chỉ tiếp xúc với khách hàng ở pha đầu tiên nên phần mềm ko đáp ứng được hết các yêu cầu của khách hàng.

+Chi phí phát triển dự án tương đối lớn.

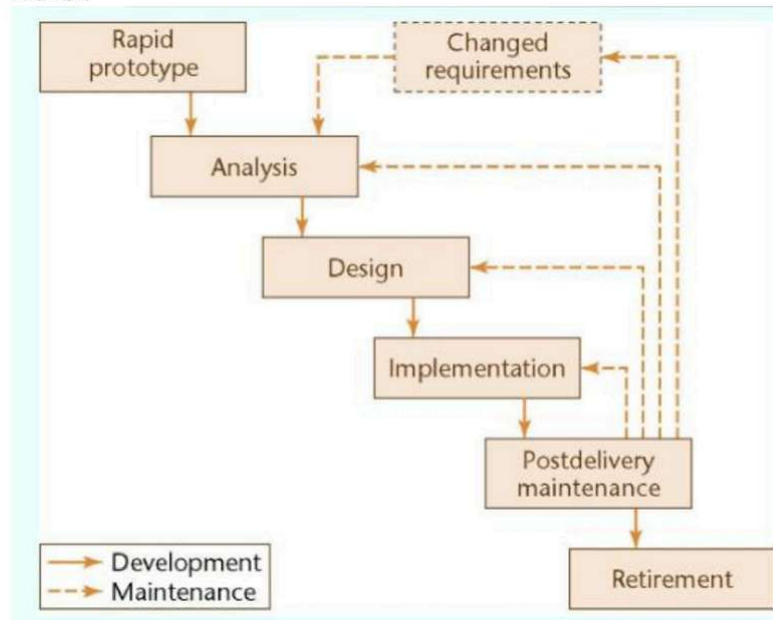
60. Mô hình vòng đời phần mềm kiểu thác nước thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

- Mô hình thác nước chỉ nên sử dụng khi mà đội dự án đã có kinh nghiệm làm việc, bởi mô hình này đòi hỏi sự chính xác ngay từ đầu.
- Hợp với những dự án mà khách hàng xác định được yêu cầu cụ thể, chính xác ngay từ đầu và ít có khả năng thay đổi
- Nên áp dụng mô hình thác nước với những dự án có fixed scope và fixed-price contract
- Thời gian phát triển dự án không gấp rút

61. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu bản mẫu nhanh?

Trả lời:



- Ưu điểm :
- +Tốc độ phát triển phần mềm nhanh
- +2 bên dễ hiểu nhau hơn (vì mô hình này không cần viết đặc tả)
- Nhược điểm:
- +đến cuối mô hình mới sửa => có nhiều nguy cơ xảy ra lỗi hơn.
- + chỉ phù hợp với những dự án đơn giản.

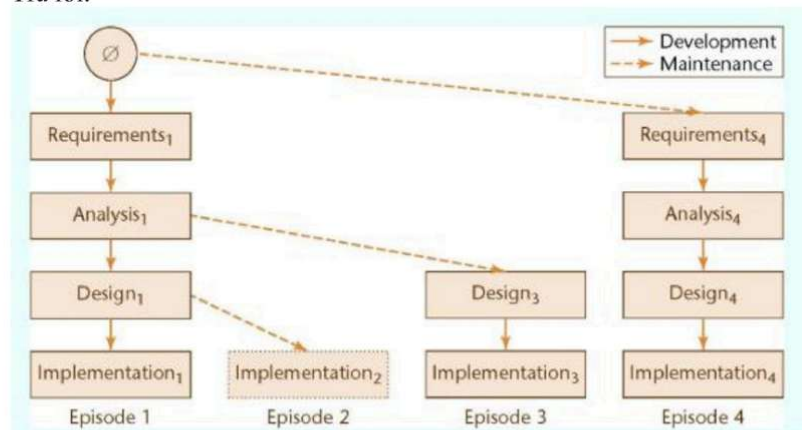
62. Mô hình vòng đời phần mềm kiểu bản mẫu nhanh thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

-Phù hợp với dự án đã được quyết định ngay từ đầu ít có khả năng thay đổi. Các hệ thống không quá lớn không có quá nhiều yêu cầu đặc tả lúc ban đầu. Các tính năng phải rõ ràng để không cần đặc tả mà vẫn có thể làm được.

63. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu lặp và tăng trưởng?

Trả lời:



-Ưu điểm:

- +Mỗi bước lặp đều có test workflow có thể phát hiện và sửa lỗi sớm
- +Thiết kế kiến trúc ngay từ đầu, mô hình theo modul ngay từ đầu, và việc tính toán tránh lỗi khi kết hợp đc tính toán trước
- +Có thể có phiên bản dùng được sản phẩm ngay từ đầu
- +Khách hàng có thể dựa vào phần đã hoàn thành để nêu chính xác yêu cầu trong những modul sau

-Nhược điểm:

- +Cần lên kế hoạch và thiết kế tốt
- +Tổng chi phí thực hiện cao

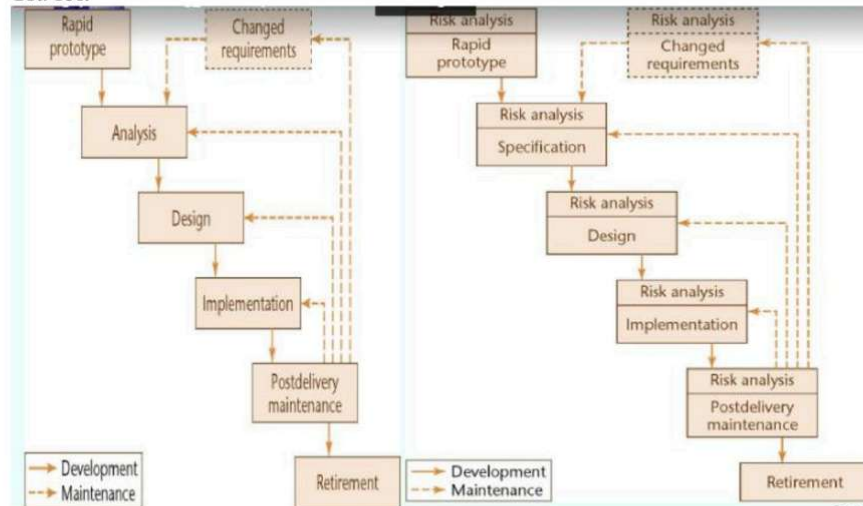
64. Mô hình vòng đời phần mềm kiểu lặp và tăng trưởng thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

- Phù hợp với những dự án đã nêu rõ mô hình kiến trúc ngay từ ban đầu và có đặc tả bản đầu.
- Mô hình này đã được sử dụng hiệu quả cho một số phần mềm vừa và nhỏ, có dung lượng công việc từ 9 man-month cho đến 100 man-year. Mô hình này thực sự có lợi khi yêu cầu của khách hàng không rõ ràng và hay thay đổi.

65. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu xoắn ốc?

Trả lời:



-Ưu điểm:

- +Tốt cho các hệ phần mềm quy mô lớn
- +Dễ kiểm soát mức độ mạo hiểm ở từng mức tiến hóa
- +Đánh giá thực tế hơn như một quy trình làm việc, bởi vì những vấn đề quan trọng đã được phát hiện sớm hơn

-Nhược điểm:

- +Chưa sử dụng rộng rãi
- +Yêu cầu thay đổi thường xuyên dẫn đến lặp vô hạn
- +Phức tạp và không thích hợp với các dự án nhỏ và ít rủi ro hơn
- +Chi phí cao và mất nhiều thời gian để hoàn thành dự án
- +Quản lý dự án cần có kỹ năng tốt để quản lý dự án, đánh giá rủi ro kịp thời

66. Mô hình vòng đời phần mềm kiểu xoắn ốc thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

- Mô hình này thường được sử dụng cho các ứng dụng lớn và phần mềm nội bộ được xây dựng theo các giai đoạn nhỏ hoặc theo các phân đoạn

67. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu tiến trình linh hoạt?

Trả lời:

-Nội dung :

- +Trích chọn các story của sản phẩm: Ước lượng thời gian và chi phí, Chọn story tiếp theo để phát triển, Mỗi story được chia nhỏ thành các task,Viết các test case cho các task trước khi cài đặt
- +Phát triển mỗi story: Không có pha đặc tả,Thiết kế linh hoạt và có thể thay đổi theo yêu cầu của khách hàng, Luôn có đại diện của khách hàng trong team, Lập trình theo cặp, Liên tục tích hợp các task
- +Sử dụng phương pháp họp đứng (stand-up meeting): Toàn bộ team đứng vòng tròn và nhìn thấy được nhau, Thời gian họp cố định hàng ngày, nhưng không quá 15p, Họp để thấy vấn đề, nếu có, chứ không giải quyết vấn đề, Lần lượt mỗi người trả lời các câu hỏi giống nhau
- +Chiến lược:Mỗi story chỉ phát triển liên tục và phải hoàn thiện sau 2-3 tuần, Cứ sau 3 tuần hoàn thành một bước lặp và bàn giao tính năng mới cho khách hàng, Sử dụng kỹ thuật timeboxing để quản lí thời gian.

→ mô hình này yêu cầu cố định thời gian, không cố định tính năng của sản phẩm

-Ưu điểm:

- +Tích kiệm được thời gian (biết được tình trạng của dự án) vì dùng phương pháp họp đứng
- +có thời gian biểu rõ ràng , toàn đội active (do dùng kỹ thuật timeboxing)
- +Cập nhật được liên tục cho khách hàng
- +Dễ dàng bổ sung thay đổi kết cấu
- +Quy tắc tối thiểu, tài liệu dễ hiểu dễ sử dụng
- +Các chức năng được xây dựng nhanh chóng rõ ràng dễ quản lý.

Nhược điểm:

- + nếu sử dụng pp họp đứng thì chỉ biết tình trạng nhưng chưa giải quyết được vấn đề
- +Không thích hợp xử lý các phụ thuộc phức tạp có nhiều rủi ro về tính bền vững
- +Phụ thuộc nhiều vào sự tương tác giữa khách hàng
- +Chuyển giao công nghệ giữa các thành viên trong nhóm khá khó khăn do thiếu tài liệu.
- +nếu dùng kỹ thuật time boxing thì phải có giải pháp tốt để chia các part để 3 tuần hoàn thiện một tính năng =>Cần một team có kinh nghiệm

68. Mô hình vòng đời phần mềm kiểu tiến trình linh hoạt thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

- Có thể sử dụng cho bất cứ dự án nào nhưng cần sự tham gia và tính tương thích với khách hàng
- Phù hợp khi khách hàng yêu cầu chức năng sẵn sàng trong thời gian ngắn

69. Trong mô hình tiến trình liên hoạt, luôn có đại diện của khách hàng trong nhóm phát triển thì có ưu điểm gì?

Trả lời:

-Có thể cập nhật các nội dung của khách hàng sát sao và đúng với yêu cầu của khách hàng nhất. Sau mỗi khi thực hiện trong story với một số chức năng có thể thực hiện hỏi độ hài lòng của khách hàng về sản phẩm, thực hiện cập nhật và sửa chữa theo yêu cầu của khách hàng ngay khi thực hiện hoàn thành sản phẩm

70. Nêu ưu điểm, nhược điểm của mô hình nhóm code bình đẳng ((Democratic Team)?

Trả lời:

-Ưu điểm:

+Các thành viên nắm chắc phần code của mình
+Khả năng code mạnh, nhất là giải quyết dự án khó
+ Do mô hình này 1 người luôn khuyến khích người khác tìm ra lỗi của mình nên việc tìm ra lỗi trong pm sẽ nhanh hơn, chất lượng phần mềm được cải thiện.

-Nhược điểm:

+Việc tự test code của bản thân thường không có hiệu quả
+Khó khăn về mặt quản lý
+Đội phát triển một cách tự nhiên
+ Những người có kinh nghiệm thường không thoải mái khi để những người non kinh nghiệm kiểm tra code của mình
+ Do không có sự cạnh tranh, thăng tiến nên thường thích hợp ở những môi trường nghiên cứu

71. Mô hình nhóm code bình đẳng thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

-Mô hình nhóm code bình đẳng thì phù hợp với những dự án kiểu nghiên cứu, giải quyết những vấn đề khó do tính bình đẳng và hỗ trợ hết sức, không có cạnh tranh, thăng tiến, không có leader thực sự trong nhóm.

72. Nêu ưu điểm, nhược điểm của mô hình nhóm code có chef?

Trả lời:

-Ưu điểm:

+ Cần ít nhân lực (khoảng 6 người), mỗi người có 1 công việc được phân công rõ ràng => Giảm chi phí, thời gian, đảm bảo chất lượng sản phẩm.
+Có khả năng quản lý
+Phân hóa công việc tốt
+Công việc rõ ràng
+Test và kiểm thử rất tốt

- Nhược điểm:
- +Tốn nhiều chi phí

73. Mô hình nhóm code có chef thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

Mô hình có Chief thì phù hợp với những dự án nghiên cứu hoặc trong những trường hợp có những vấn đề khó cần sự giải quyết hợp lực của 1 nhóm tương tác lẫn nhau để đưa ra giải pháp, phù hợp những dự án cần tính chuyên môn hóa cao sẽ giúp các phần được hoàn thành một cách trơn tru nhất có thể giúp tối ưu hóa kinh tế và thời gian làm việc.

74. Nêu ưu điểm, nhược điểm của kỹ thuật pair programming?

Trả lời:

- Ưu điểm :

- + Thời gian hoàn thành công việc nhanh, thời gian cài đặt và tích hợp chỉ trong vài giờ
 - + Khi một người rời khỏi vị trí công việc, người còn lại vẫn có đủ hiểu biết để tiếp tục với người ghép cặp mới, tránh tình trạng phải cài đặt lại từ đầu.
 - + Làm việc trong 1 cặp sẽ giúp người non kinh nghiệm học hỏi được nhiều từ người cùng cặp giàu kinh nghiệm hơn.
 - + Tất cả máy tính của các cặp sẽ được đặt ở giữa một phòng lớn, do đó tăng tính sở hữu code theo nhóm – đặc trưng tích cực của egoless team (đội bình đẳng)
- Nhược điểm :
- + Đòi hỏi những khoảng thời gian lớn làm việc liên tục
 - + Không hiệu quả trong trường hợp có những cá nhân ngại ngừng, kiêu ngạo hoặc cả 2 người đều thiếu kinh nghiệm

75. Kỹ thuật pair programming thì phù hợp với những dự án có đặc điểm gì?

Trả lời:

- Phù hợp với các dự án có độ phức tạp cao và nhất là cần nhiều thời gian để tìm hiểu.
- Khi có 1 task phức tạp, hoặc một PR quá lớn.
- Khi coaching
- Khi fixing
- Hoặc đơn giản là muốn nâng cao trình độ các member trong team lên

76. Nêu ưu điểm, nhược điểm của kỹ thuật time boxing?

Trả lời:

-Ưu điểm:

- +có thời gian biểu rõ ràng . Toàn đội đều được hoạt động với nhau
- +Tăng năng suất của quá trình phát triển phần mềm.
- +Rút ngắn thời gian phát triển phần mềm (vì cứ sau 2-3 tuần là sẽ đóng gói 1 tính năng)
- Nhược điểm:
 - +Quản lý dự án trở nên phức tạp hơn (vì làm như thế nào để chia các phần công việc trong 2-3 tuần như vậy hoàn thành 1 tính năng)
 - +Công việc sẽ bị cắt giảm bớt nếu không hoàn thành trong khoảng thời gian được định sẵn. do timeboxing chỉ yêu cầu thời gian cố định chứ không có chỉ tiêu cố định nên chỉ thích hợp với các dự án nhỏ.

77. Nêu ưu điểm, nhược điểm của kĩ thuật stand up meeting?

Trả lời:

- Ưu điểm:
 - +Rút ngắn thời gian phát triển phần mềm (vì ta biết được tình trạng của dự án)
- Nhược điểm:
 - +Không giải quyết được vấn đề.
 - +Chỉ phù hợp với dự án nhỏ

78. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng LOC?

Trả lời:

- Ưu điểm:
 - +Dễ dàng đo lường khi dự án được hoàn thành
 - +Được sử dụng rộng rãi.
 - +Các hoạt động cải tiến liên tục tồn tại đối với các kỹ thuật ước lượng.
- Nhược điểm:
 - +Không thể đo kích thước của đặc điểm kỹ thuật
 - +Nó chỉ đặc trưng cho một chế độ xem cụ thể về kích thước, cụ thể là chiều dài, nó không tính đến chức năng hoặc độ phức tạp.
 - +Thiết kế phần mềm không tốt có thể gây ra quá nhiều dòng mã.
 - +Nó phụ thuộc vào ngôn ngữ lập trình
 - +Người dùng không thể hiểu nó một cách dễ dàng.

79. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng FFP?

Trả lời:

- FFP=File+Flow+Proccess=Size
- Code: $c=b*S$
- Ưu điểm:
 - +Chính xác hơn khi xác định được dự án này cần xử lý bao nhiêu file, tiến trình, luồng/
 - +Áp dụng ở thời điểm sớm hơn LOC (ở cuối pha thiết kế)

- Nhược điểm:
- +file có thể ít hoặc nhiều thông tin nhưng vẫn chưa phản ánh được độ phức tạp của phần mềm.
- +Phụ thuộc vào hằng số b (phụ thuộc vào kinh nghiệm trong từng công ty)

80. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng Function Point?

- Ưu điểm:
- +Nó có thể được áp dụng sớm trong vòng đời phát triển phần mềm.
- +Nó độc lập với ngôn ngữ lập trình, công nghệ, kỹ thuật.
- +Chúng có thể được tính toán sớm và thường xuyên.
- Nhược điểm:
- +Là một phương pháp tốn thời gian.
- +Nó có độ chính xác thấp trong việc đánh giá do có liên quan đến một phán đoán chủ quan (dựa vào chuyên gia).
- +Nó được thực hiện sau khi tạo ra các thông số kỹ thuật thiết kế.
- +Ít dữ liệu nghiên cứu có sẵn hơn

81. Nêu ưu điểm, nhược điểm của phương pháp ước lượng phần mềm bằng COCOMO?

- Ưu điểm:
- +Nó hoạt động dựa trên dữ liệu lịch sử và cung cấp các chi tiết chính xác hơn.
- +Dễ dàng thực hiện với nhiều yếu tố khác nhau. Người ta có thể dễ dàng hiểu nó hoạt động như thế nào.
- +Dễ dàng ước tính tổng chi phí của dự án.
- +Các trình điều khiển rất hữu ích để hiểu tác động của các yếu tố khác nhau ảnh hưởng đến khủng hoảng dự án.
- Nhược điểm:
- +Nó bỏ qua các vấn đề phần cứng cũng như mức doanh thu cá nhân.
- +Nó bỏ qua tất cả các tài liệu và yêu cầu.
- +Nó chủ yếu phụ thuộc vào yếu tố thời gian.
- +Nó hạn chế độ chính xác của chi phí phần mềm.
- +Nó cũng bỏ qua các kỹ năng, sự hợp tác và kiến thức của khách hàng
- +Nó đơn giản hóa tác động của các khía cạnh an toàn hoặc bảo mật.

82. Tại sao trong mô hình tiến trình linh hoạt, không cần có pha đặc tả?

- Trả lời:
- Tiến trình linh hoạt là quá trình trích chọn các story của sản phẩm và mỗi story được chia nhỏ thành các task. Phát triển mỗi story được thiết kế linh hoạt và có thể thay đổi theo yêu cầu của khách hàng. Việc phần mềm chạy được quan trọng hơn là những tài liệu chi tiết. Đồng thời luôn có đại diện khách hàng trong

nhóm phát triển nên những tiêu chí của khách hàng sẽ được trao đổi, thực hiện, thay đổi trong lúc cài đặt nên không cần pha đặc tả.

83. Tại sao trong nhóm walkthrough và inspection, luôn phải có đại diện của workflow tiếp theo?

Trả lời:

Vì người đại diện cho nhóm workflow tiếp theo sẽ chuyển những thông tin công việc từ workflow trước về thông tin công việc của mình. Mà nhóm walkthrough và inspection lại là nhóm đảm bảo tài liệu luồng công việc trước phải luôn có sẵn

84. Nếu nhóm SQA phát hiện ra ít lỗi, thì có thể kết luận nhóm code giỏi hay nhóm SQA kém? Tại sao?

Trả lời:

- Do sản phẩm làm là thực tế mà mức độ phức tạp cũng khác nhau điều này rất khó khẳng định.

=>Đối với các phần mềm nhỏ, nếu SQA phát hiện ra ít lỗi thì có thể kết luận nhóm code giỏi.

=>Đối với các phần mềm lớn (có khả năng rất nhiều lỗi) nếu SQA phát hiện ra ít lỗi thì có khả năng là nhóm SQA kém, không bao quát được cả trường hợp cần thử cho các phần mềm.

85. Tại sao nói inspection và walkthrough là hướng tài liệu, mà không phải hướng vào người tham gia?

Trả lời:

-Bởi vì mục đích cuối cùng của inspection và walkthrough đó là kiểm tra một cách cẩn thận tài liệu(chẳng hạn như một tài liệu đặc tả, một tài liệu thiết kế hoặc một mã tạo tác) để chúng được hoàn thiện và luôn có sẵn

86. Quality assurance thì khác gì với testing?

Trả lời:

	Quality Assurance	Testing
Định nghĩa	Các hoạt động được thiết kế để đảm bảo phần mềm tương ứng với đặc điểm kỹ thuật	Quá trình khám phá một hệ thống để tìm ra các khiếm khuyết
Tập trung vào	Kiểm soát chất lượng và đáp ứng các yêu cầu	Kiểm tra hệ thống và tìm lỗi

Hướng vào	Hướng vào quá trình	Hướng vào sản phẩm
Loại hoạt động	Ngăn ngừa	Sửa chữa
Mục tiêu	Để đảm bảo chất lượng	Để kiểm soát chất lượng

87. Tại sao nói function point chịu ảnh hưởng chủ quan của các chuyên gia?

Trả lời:

-Trong công thức ước lượng của function point thì ngoài số yêu cầu ra, còn lại tất cả các tiêu chí, các thành phần dùng để đánh giá, ước lượng đều có 1 số điểm nhất định cho nó. Và số điểm này cao hay thấp là phụ thuộc và sự đánh giá của các chuyên gia.

88. COCOMO tính đến nhiều tiêu chí hơn hay là function point? Giải thích?

Trả lời:

-Function point là tích của UFP, được tính bằng tổng của 5 số là Inp: số item input, Out: số item output, Inq: số inquiries (yêu cầu), Maf: số master file (file chủ), Inf: số interface (giao thức) và TCF, tổng điểm số độ ảnh hưởng của 14 yếu tố kỹ thuật trên thang từ 0 đến 5. Có thể thấy rằng các tiêu chí đều thuộc mặt kỹ thuật

-COCOMO được tính toán bởi tham số KDSI và tích của 15 trọng số tương ứng với độ khó của các tiêu chí đánh giá chi phí, với các tiêu chí được xét đến thuộc một trong 4 loại: thuộc tính của sản phẩm, thuộc tính máy tính, thuộc tính con người và thuộc tính dự án. Ta có thể dễ dàng thấy rằng các tiêu chí này thuộc nhiều phạm trù hơn FP

89. SW development multiplier của COCOMO thì khác gì TCF của function point?

Trả lời:

-TCF của Function point: dựa vào tổng của điểm số độ ảnh hưởng cho 14 yếu tố của dự án, giá trị trong khoảng 0 (không ảnh hưởng) đến 5 (ảnh hưởng mạnh) và được người đánh giá xác định

-Software development multiplier: dựa vào hằng số cho sẵn được gán cho độ khó của 15 tiêu chí đánh giá chi phí, với độ khó chạy từ rất thấp đến cực khó (0.7->1.65)

90. TCF của function point thì khác gì hằng số b của FFP?

Trả lời:

- TCF của Function point: dựa vào tổng của điểm số cho 14 yếu tố của dự án, giá trị của mỗi yếu tố trong khoảng 0 đến 5, và giá trị của TCF nằm trong khoảng từ 0.65 đến 1.35
- Hằng số b của FFP: hằng số đo độ hiệu quả của công việc của tổ chức đó, được ước lượng bằng các dự án trước và khác nhau đối với mỗi tổ chức

91. Tại sao nguyên lý Dijkstra lại đúng?

Trả lời:

- Dijkstra nói rằng rất dễ để chứng minh một phần mềm có lỗi, nhưng không thể chứng minh được một phần mềm không có lỗi. Đơn giản là vì việc có thể nghĩ được tất cả các test case trong 1 dự án lớn là điều không thể dù là với programmer hay tester. Điều quan trọng là nó áp dụng được trong việc giải quyết các vấn đề hiện tại của khách hàng, đó đã được coi là một sản phẩm thành công.

92. Tại sao luật Brook lại đúng?

Trả lời:

- Luật Brook nói rằng thêm một người vào dự án đôi khi sẽ làm nó bị chậm hơn so với tiến độ ban đầu => Luật này là đúng vì với những dự án lớn, tài liệu nghiên cứu nhiều thì việc thêm 1 người mới đồng nghĩa với việc phải nói cho họ biết những kiến thức mà nhóm code trước đó đã tìm hiểu được đồng thời phải cho họ biết những vấn đề đang gặp phải và cũng cần thời gian để họ thích nghi được với nhóm code hiện tại.

93. Người ta áp dụng luật Miller trong CNPM như thế nào?

Trả lời:

- Định luật Miller : Tại mỗi thời điểm, người ta chỉ có thể tập trung vào tối đa khoảng 7 vấn đề
- => Áp dụng luật Miller trong CNPM → để xử lý các vấn đề lớn, sử dụng phương pháp làm mịn từng bước: ta phải tập trung xử lý các việc quan trọng trước, các việc ít quan trọng hơn xử lý sau → gọi là tiến trình tăng trưởng

94. Phát triển phần mềm thì khác gì sản xuất phần mềm?

Trả lời:

- Việc phát triển một phần mềm cần một nguồn lực rất lớn, bắt đầu từ khâu đầu tiên là phân tích yêu cầu, lập bản thiết kế cho tới công đoạn bảo hành sau sử dụng để đảm bảo các ý tưởng của khách hàng trở thành những ứng dụng thực tế và hữu ích nhất còn sản xuất phần mềm thì đơn thuần hơn vì thường được đặt theo yêu cầu nên các bước phân tích thiết kế sẽ được giảm lược đi, chỉ tập trung chính vào phần code những gì đã được bên khách hàng đã đặt trước.

95. Test trường hợp sai kiểu dữ liệu đầu vào thì thuộc thể loại test gì?

Trả lời:

-Thuộc thể loại Negative testing (Negative testing được sử dụng để kiểm tra các phần mềm hoạt động khi nó nhận được thông tin không chính xác hoặc không hợp lệ)

Câu hỏi khác liên quan:

1. Phần mềm là gì? Nêu đặc trưng của nó. Có những loại ngôn ngữ nào để phát triển phần mềm?

- Phần mềm là sản phẩm do các nhà phát triển phần mềm thiết kế và xây dựng
- Đặc trưng: Phần mềm không hao mòn, phần mềm được phát triển chức năng không phải là sản xuất, phần mềm rất phức tạp chi phí thay đổi rất lớn.
- Các ngôn ngữ: java .c,c++....

2. Phân loại phần mềm và nội dung cơ bản mỗi loại.

- Phần mềm hệ thống: (hệ điều hành, phần mềm vận hành thiết bị,...) giúp vận hành phần cứng
- Phần mềm ứng dụng: cung cấp công cụ hỗ trợ lập trình viên trong khi viết chương trình và phần mềm bằng các ngôn ngữ khác nhau.
- Các loại khác : virus được viết để chạy với mục đích xấu.

3. Định nghĩa kỹ nghệ phần mềm? Những yếu tố chủ chốt trong kỹ nghệ phần mềm là gì?

- KNPM: là sự áp dụng một cách tiếp cận có hệ thống, có kỉ luật, và định lượng được cho việc phát triển, hoạt động và bảo trì phần mềm.
- KNPM bao trùm kiến thức, các công cụ và các phương pháp cho việc định nghĩa yêu cầu phần mềm, và thực hiện các tác vụ thiết kế, xây dựng kiểm thử, bảo trì phần mềm.

4. Chất lượng phần mềm là gì? Các tiêu chí của chất lượng phần mềm.

- Chất lượng phần mềm là sự đáp ứng các yêu cầu chức năng, sự hoàn thiện và các chuẩn(đặc tả) được phát triển, các đặc trưng mong chờ từ mọi phần mềm chuyên nghiệp(ngầm định)

Các tiêu chí:

1. tính bảo trì
2. độ tin cậy
3. tính hiệu quả
4. tính khả dụng

5. Có các dạng bảo trì nào? Nêu và phân biệt.

4 loại:

- bảo trì để tu chỉnh: là bảo trì khắc phục những khiếm khuyết co trong phần mềm.
- bảo trì để thích nghi: là tu chỉnh phần mềm theo thay đổi của môi trường bên ngoài nhằm duy trì, thích nghi và quản lí phần mềm theo vòng đời của nó.
- bảo trì để hoàn thiện : là việc tu chỉnh phần mềm theo các yêu cầu ngày càng hoàn thiện hơn, đầy đủ hơn, hợp lí hơn.
- bảo trì để phòng ngừa: là công việc tu chỉnh chương trình có tính đến tương lai của phần mềm đó sẽ mở rộng và thay đổi ntn.

6. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu ổn định và đồng bộ hóa?

-Ưu điểm:

+ những người phát triển có thể sớm nhìn thấy sự hoạt động của phần mềm và có thể hiệu chỉnh các yêu cầu, có thể là ngay trong quá trình các thành phần được xây dựng.

+ Mô hình này có thể áp dụng ngay cả trong trường hợp các đặc tả ban đầu không hoàn thiện.

-Nhược điểm: chỉ phù hợp các dự án không có thay đổi trong đặc tả.

7. Mô hình vòng đời phần mềm kiểu ổn định và đồng bộ hóa thì phù hợp với những dự án có đặc điểm gì?

-Hợp với hệ thống lớn có thể phân chia thành nhiều thành phần, ít rủi ro.

-Phần mềm lấy đặc trưng của nhu cầu khách hàng được liệt kê theo thứ tự ưu tiên

-Phần mềm được đóng gói.

8. Nêu ưu điểm, nhược điểm của mô hình vòng đời phần mềm kiểu mã nguồn mở?

*Ưu điểm:

- tính tái sử dụng: bạn có thể tạo các thành phần (đối tượng) một lần và dùng chúng nhiều lần sau đó.

- khả năng tái sử dụng đối tượng có tác dụng giảm thiểu lỗi và các khó khăn trong việc bảo trì, giúp tăng tốc độ thiết kế và phát triển phần mềm.

- Phương pháp hướng đối tượng giúp chúng ta xử lý các vấn đề phức tạp trong phát triển phần mềm và tạo ra các thể hệ phần mềm có khả năng thích ứng và bền chắc.

- Dễ dàng mở rộng, nâng cấp

*Nhược điểm:

- có những hạn chế như báo cáo thất bại (các báo cáo về hành vi không chính xác quan sát được), người dùng không có quyền truy cập vào mã nguồn, vì vậy họ không thể gửi báo cáo lỗi (báo cáo đưa ra chỗ nào mã nguồn không chính xác và làm thế nào để sửa chữa nó).

- Rất khó để hình thành việc phát triển mã nguồn mở của một sản phẩm phần mềm để sử dụng trong một tổ chức thương mại.

- Mô hình có thể dẫn đến sự phát triển tùy tiện của các pha, và làm tùy tiện không có kỷ luật

9. Mô hình vòng đời phần mềm kiểu mã nguồn mở thì phù hợp với những dự án có đặc điểm gì?

- Phù hợp với các hệ thống lớn: Phương pháp hướng đối tượng không chia bài toán thành các bài toán nhỏ mà tập trung vào việc xác định các đối tượng, dữ liệu và hành động gắn với đối tượng và mối quan hệ giữa các đối tượng.

- Hỗ trợ sử dụng lại mã nguồn

10. Tại sao trong mô hình tiến trình linh hoạt, không cần có pha đặc tả?

- Tiến trình linh hoạt (Agile Process) không có pha đặc tả do pha phân tích trong tiến trình linh hoạt không được chú trọng. Việc phần mềm chạy được quan trọng hơn là những tài liệu chi tiết. Đồng thời luôn có đại diện khách hàng trong nhóm phát triển nên những tiêu chí của khách hàng sẽ được trao đổi, thực hiện, thay đổi trong lúc cài đặt.

11. Nêu ưu, nhược điểm của phương pháp ước lượng phần mềm bằng COCOMO ?

Gồm 3 loại COCOMO :

*) COCOMO Cơ bản (Basic) :

- Ưu điểm : Áp dụng tốt để tính nhanh và sơ bộ chi phí cho các dự án nhỏ và vừa

- Nhược điểm : Bị giới hạn chức năng do không cân nhắc đến các yếu tố như ràng buộc phần cứng, chất lượng nhân sự, kinh nghiệm, trình độ kỹ thuật và công cụ sử dụng

*) COCOMO Trung gian (Intermediate)

- Ưu điểm : Được áp dụng cho gần như toàn bộ dự án ở những cài đặt đơn giản và thô sơ. Nó cũng có thể được áp dụng để tính chi phí ở mức các thành phần của phần mềm

- Nhược điểm : Sản phẩm có quá nhiều thành phần sẽ khó tính được bằng Intermediate COCOMO

*) COCOMO Chi tiết (Detailed)

- Ưu điểm : Dễ hiểu, trực quan, đặc biệt hữu ích cho người ước lượng hiểu được ảnh hưởng của những yếu tố khác nhau đến chi phí sản phẩm

- Nhược điểm : Thành công của pp phụ thuộc vào mô hình áp dụng với yêu cầu của khách hàng.

12. Ý nghĩa của scenario và ngoại lệ ?

- Từ scenario ta làm rõ được chức năng của use case và sự tương tác giữa người dùng/ người quản trị với hệ thống.

- Từ scenario ta cũng xác định được ngoại lệ và cách xử lý ngoại lệ của hệ thống.

13. Ý nghĩa của sơ đồ tuần tự ?

Sơ đồ tuần tự thể hiện sự tương tác giữa các lớp trong hệ thống với nhau và với người dùng.

14. Ý nghĩa của sơ đồ lớp ?

- Mỗi quan hệ giữa các đối tượng trong hệ thống sẽ được thể hiện qua mối quan hệ giữa các lớp.

- Mô tả 1 hướng nhìn tĩnh về 1 hệ thống bằng các lớp, các thuộc tính, phương thức của lớp và mối liên hệ giữa chúng.

15. Thế nào là lớp giao diện ? Lớp này thường tương tác với các lớp nào ?

- Lớp giao diện là lớp có chức năng tương tác giữa môi trường bên ngoài và phần mềm. Lớp giao diện thường là giao diện vào ra với người dùng

- Lớp giao diện tương tác với lớp điều khiển để trao đổi thông tin giữa người dùng và hệ thống.

16. Thế nào là lớp điều khiển, lớp này tương tác với những lớp nào?

*Lớp điều khiển (còn gọi là lớp control)

- Dùng để mô hình các tính toán và thuật toán phức tạp trong hệ thống.

- Có thể chỉ dùng 1 lớp điều khiển cho các hệ thống đơn giản, mỗi phương thức là 1 hàm xử lý, tính toán độc lập.

- Lớp điều khiển tương tác với các lớp thực thể (model) và lớp biên (view) làm nhiệm vụ tính toán và xử lý thông tin

17. Thế nào là lớp thực thể, lớp này tương tác với những lớp nào?

Lớp thực thể (còn gọi là lớp model).

- Dùng để biểu diễn dữ liệu để xử lý, trao đổi giữa các đối tượng trong hệ thống.

- Thường chỉ có các thuộc tính và các phương thức truy nhập (get/set).

- Lớp thực thể tương tác với lớp điều khiển (control) để cung cấp và lưu trữ thông tin.

18. Scenario và sơ đồ tuần tự có liên hệ gì đến nhau?

- Sơ đồ tuần tự minh họa các đối tượng tương tác với nhau ra sao.

- Một sơ đồ tuần tự nêu lên sự tương tác trong một scenario: Sự tương tác xảy ra tại một thời điểm nào đó trong quá trình thực thi hệ thống.

- Ngược lại, scenario chính là nội dung của sự tương tác của các đối tượng được thể hiện trong sơ đồ tuần tự.

19. Scenario và sơ đồ lớp có quan hệ gì đến nhau?

- Một nhóm đối tượng có chung 1 số thuộc tính và phương thức sẽ tạo thành 1 lớp. Mỗi tương tác giữa các đối tượng trong hệ thống được biểu hiện qua mỗi tương tác giữa các lớp.

- Sơ đồ lớp được tạo thành từ các lớp (bao gồm các thuộc tính và phương thức), và các mối quan hệ của nó.

- Mỗi lớp trong 1 sơ đồ sẽ bao gồm đầy đủ các thuộc tính và các phương thức cùng mối quan hệ của chúng đã được thể hiện trong scenario.

- Ngược lại, scenario đưa ra những thuộc tính, phương thức mà sơ đồ cần thể hiện.

20. Việc trích lớp và xây dựng các lớp là của pha thiết kế, tại sao người ta làm nó ngay trong pha phân tích?

- Là do bước đầu tiên trong quá trình phân tích là xác định các lớp. Bởi vì mỗi lớp là một kiểu của mô-đun, việc phân tích mô-đun được thực hiện trong suốt quá trình phân tích. "Because a class is a type of module, the modular decomposition has been performed during the analysis workflow ???"

21. Kỹ thuật trích danh từ dùng để trích các lớp nào? Có thể dùng để trích lớp biên và lớp điều khiển được không?

- Kỹ thuật này được dùng để trích các lớp thực thể, không thể dùng để trích các lớp biên và lớp điều khiển vì trong phân kịch bản của 1 use case chỉ mô tả tương tác giữa người dùng và hệ thống mà không nói đến cách thức hệ thống xử lý, điều khiển các chức năng.

22. Trình bày kỹ thuật trích lớp điều khiển? Số lượng lớp điều khiển nhiều hay ít thì tốt?

Kỹ thuật trích các lớp điều khiển:

- Toàn bộ hệ thống dùng chung một lớp điều khiển.
- Mỗi modul dùng riêng một lớp điều khiển.

23. Ý nghĩa của sơ đồ trạng thái hữu hạn? Nó biểu diễn trạng thái của cả hệ thống, của lớp hay của phương thức?

Ý nghĩa: Mô tả các sự kiện và hoạt động của từng hệ thống, lớp hoặc phương thức, cụ thể trong từng thao tác và gắn với phương thức/ trừ trường hợp xảy ra với từng đối tượng.

Được sử dụng cho cả hệ thống, các lớp và phương thức.

24. Ý nghĩa thẻ CRC, dùng thẻ CRC với lớp biên và lớp điều khiển được không? Có cần thiết không?

Ý nghĩa thẻ CRC : Được dùng để mô hình hóa quan hệ giữa các lớp

- ☐ C : class. Biểu diễn tên lớp.
- ☐ R : responsibility. Trách nhiệm của lớp.
- ☐ C : collaboration. Quan hệ của lớp.

Có cần thiết vì nhờ CRC ta mới vẽ được MVC

25. Thiết kế kiến trúc cần sơ đồ quan hệ các lớp (Communication Diagram)

26. Thiết kế chi tiết cần Sơ đồ cần dùng

Sequence Diagram (tuần tự), Class Diagram (Sơ đồ lớp dạng ô vuông có thể điền thuộc tính và phương thức)

27. Làm thế nào để trích các lớp? Cần dùng các sơ đồ nào của UML?

Kỹ thuật để trích các lớp :

- Mô tả hoạt động của ứng dụng trong một đoạn văn.
- Trích các danh từ xuất hiện trong đoạn văn đó, coi như là một ứng cử viên cho lớp thực thể.
- Xét duyệt từng danh từ và đề xuất nó là lớp thực thể hay là thuộc tính của lớp thực thể.
- Sơ đồ cần dùng : Sơ đồ quan hệ các lớp.

28. Sơ đồ lớp và sơ đồ cộng tác có gì khác nhau?

- Sơ đồ lớp chỉ ra cấu trúc bên trong của các lớp, bao gồm thuộc tính, phương thức, các lớp quan hệ với nhau theo nhiều dạng thức : kế thừa, liên kết...
- Sơ đồ cộng tác chỉ ra sự tác động giữa các đối tượng với nhau, xác định các đối tượng và quan hệ giữa chúng, thể hiện sự tương tác qua những mũi tên theo một dòng chảy thông điệp giữa các đối tượng.

29. Mỗi trạng thái của sơ đồ trạng thái ứng với một lớp hay một phương thức? Tại sao?

Ứng với 1 phương thức vì:

- Trạng thái là một kết quả của các hoạt động trước đó đã được đối tượng thực hiện và nó thường xác định qua giá trị của các thuộc tính cũng như các nối kết của đối tượng với các đối tượng khác. Một lớp có thể có 1 thuộc tính đặc biệt

xác định trạng thái, hoặc trạng thái cũng có thể được xác định qua giá trị của các thuộc tính “bình thường” trong đối tượng.

- Mỗi sự chuyển trạng thái được ánh xạ thành 1 phương thức của lớp, mỗi hành động tác động vào trạng thái được ánh xạ vào phương thức tương ứng.

30. Trình bày nguyên lý A của phần gán phương thức cho lớp, nguyên lý này

thường dùng cho lớp loại nào?

Nguyên lý A:

-Che dấu thông tin: Các thuộc tính của lớp phải để ở dạng private -> cần các phương thức get/set tương ứng cho các đối tượng khác truy nhập vào các thuộc tính này.

-Áp dụng cho các lớp thực thể: các thuộc tính để private, mỗi thuộc tính có một cặp get/set tương ứng.

31. Trình bày nguyên lý B của phần gán phương thức cho lớp, nguyên lý này thường dùng cho lớp loại nào?

Nguyên lý B:

- Nếu có nhiều đối tượng X gọi đến một hành động k của đối tượng Y thì phương thức để thực hiện hành động k nên gán cho lớp của đối tượng Y, không nên gán cho lớp của đối tượng X.

32. Trình bày nguyên lý C của phần gán phương thức cho lớp, nguyên lý này thường dùng cho lớp loại nào?

Nguyên lý C:

- Thiết kế hướng trách nhiệm: Nếu một hành động mà không thể gán thành phương thức cho lớp khác, thì lớp của đối tượng cần thực hiện hành động đó phải chứa phương thức tương ứng của hành động đó.

33. inspection và walkthrough?

*** Inspection:**

Team Inspection thường gồm 4 người. Vd cho Inspect design gồm có:

Moderator, designer, implementer, tester.

là 1 quy trình gồm 5 bước:

1/ (Overview)Nhìn lại tổng quan toàn bộ tài liệu điều tra (lấy yêu cầu, đặc tả, thiết kế, code hoặc kế hoạch), tài liệu được phát cho các thành viên.

2/ (Preparation) các thành viên hiểu chi tiết tài liệu. Liệt kê danh sách các loại lỗi, sắp xếp theo tần suất xuất hiện. Việc này giúp mọi người tập trung vùng có nhiều lỗi.

3/ 1 người sẽ lướt qua tài liệu, và các thành viên điều tra sẽ xác định lỗi mà không sửa chúng. Và moderator của inspection team trong 1 ngày sẽ có báo cáo, bảo đảm quy trình được thực hiện đúng.

4/ (Rework) các thành viên sửa toàn bộ lỗi được chỉ ra trong báo cáo.

5/ (Follow Up) Moderator phải kiểm tra toàn bộ, bảo đảm không còn lỗi trong báo cáo và không có lỗi phát sinh. Nếu 5% tài liệu trên phải rework (bước 4) lại thì team phải điều tra lại (reinspection).

- Walkthrough:

Gồm có 4-6 người, 1 người viết đặc tả (specification), 1 người quản lý workflow phân tích, đại diện khách hàng, 1 người thuộc team thiết kế, 1 đại diện thuộc SQA (leader)

. Có 2 kiểu walkthrough : Document Driven và participants Driven

+ Document Driven: 1 người (presenter) sẽ kiểm tra toàn bộ nội dung, và những người còn lại có trách nhiệm bình luận, ngắt quãng những chỗ nghi ngờ có lỗi.

(cách hiệu quả hơn để tìm lỗi)

+ Participant Driven: mỗi thành viên chuẩn bị 1 danh sách gồm 2 cột: các chi tiết chưa rõ ràng hoặc chi tiết nghi ngờ dính lỗi. Đại diện team phân tích phải phân tích được đâu là lỗi và làm rõ cho người hỏi.

34. Người ta nói « nhóm SQA tạo ra chất lượng cho phần mềm » đúng hay sai? Tại sao?

- Sai, SQA không tạo ra chất lượng phần mềm, họ chỉ bảo đảm chất lượng phần mềm. Còn người tạo nên chất lượng phần mềm là Developer. (câu này thầy Hùng đã khẳng định)

35. Trình bày phương pháp kiểm thử hộp trắng: cơ sở phương pháp; các yêu cầu cần kiểm tra, các kỹ thuật được sử dụng.

- Kiểm thử hộp trắng (test to code, glass-box, white-box, structural, logic-driven, and path-oriented testing) là phương pháp chọn testcase dựa trên việc kiểm tra code, không phải kiểm tra bản đặc tả.

- Kỹ thuật được sử dụng:

+ Statement Coverage: Phương pháp này chạy 1 loạt các testcase song song với việc đảm bảo mỗi câu (statement) được chạy ít nhất 1 lần. Yêu cầu cần có 1 CASE tool để theo dõi câu lệnh nào đó được thực hiện bao nhiêu lần trong test. Điểm yếu của phương pháp này: nó không bảo đảm đầu ra của mọi nhánh (branch) đều được kiểm tra.

+ Branch Coverage: Chạy 1 loạt các test để bảo đảm tất cả các nhánh đều được kiểm tra ít nhất 1 lần. Và cần có 1 tool để biết được nhánh nào đã kiểm tra (hoặc chưa).

+ Path Coverage: Phương pháp hiệu quả nhất, cho phép kiểm tra tất cả các đường đi (path) của chương trình. Tuy nhiên (giả sử trong 1 chương trình có nhiều vòng lặp) thì số đường đi là rất lớn. Do vậy, vấn đề giảm bớt số đường đi và phát hiện đủ lỗi đang được nghiên cứu.

36. Trình bày phương pháp kiểm thử hộp đen: cơ sở phương pháp; các yêu cầu cần kiểm tra, các kỹ thuật được sử dụng.

- Kiểm thử hộp đen (test to specifications, black-box, behavioral, datadriven, functional, and input/output-driven testing.) là phương pháp chọn test case dựa trên tài liệu đặc tả và bỏ qua code.

- kỹ thuật được sử dụng:

+ Equivalence Testing and Boundary Value Analysis:

+ Equivalent Class (lớp tương đương): (pg521): Tất cả các testcase trong lớp tương đương đều có độ tốt như nhau. Do đó với mỗi lớp tương đương chỉ cần 1 testcase là đủ.

VD: quản lý hàng hóa có STT từ 1 đến 2000, ta có 3 khoản Equi Class: (<1); [1,1000], (>1000) và mỗi khoảng chỉ cần 1 testcase.

+ Boundary Value Analysis (phân tích giá trị biên): Để mở rộng khả năng tìm lỗi, hỗ trợ cho Equivalent Class.

vd: một tham số đầu vào có một giới hạn biên x, thì phải test ít nhất 4 trường hợp:

- 1: giá trị đầu vào bất kỳ cách xa x
- 2: giá trị đầu vào ngay trên x
- 3: giá trị đầu vào ngay dưới x
- 4: giá trị đầu vào đúng bằng x

+ Functional Testing: Những dữ liệu kiểm tra sẽ phụ thuộc vào chức năng của code artifact. Sau khi xác định được chức năng của code artifact (vd: code artifact có chức năng xác định có phải số nguyên tố hay không?), tập dữ liệu kiểm tra được sử dụng để kiểm tra các chức năng 1 cách độc lập nhau.

37. Kiểm thử đơn vị đối tượng là gì? Ai thực hiện. Các phương pháp và kỹ thuật nào được sử dụng? Kiểm tra những loại lỗi nào?

- Kiểm thử đơn vị (Unit testing) là việc kiểm thử của lập trình viên sau khi hoàn thành xong 1 code artifact. Sau khi kiểm thử đơn vị hoàn tất, code artifact được gửi cho SQA để tiếp tục kiểm tra.

- Có 2 kỹ thuật cho phép kiểm tra: Hộp trắng & Hộp đen. Như đã mô tả ở trên.

38. Giải thích khái niệm stub và driver? Chúng được sử dụng ở đâu và vì sao?

- Stub: Trong quá trình tích hợp các artifact với nhau, có những artifact(giả sử A) cần những artifact khác (vd: B,C,D) để có thể chạy được. Khi đó, B,C,D được gọi là Stub.

- Driver: Trong ví dụ trên, để kiểm tra artifact A ta cần 1 Driver và 3 Stub là B,C,D. (Giả sử) nếu B có thể tự chạy được mà không cần artifact nào, Có thể kiểm tra B với 1 Driver và 0 Stub.

- Chúng được sử dụng trong quá trình tích hợp các artifact với nhau, quá trình test tích hợp để tạo ra sản phẩm hoàn chỉnh.

39. Kiểm thử hệ thống nhằm kiểm tra cái gì? Ai thực hiện? Các phương pháp?

- (Product Testing): Diễn ra sau khi quá trình kiểm tra tích hợp được hoàn tất. Mục đích bảo đảm phần mềm hoàn tất. Thực hiện bởi SQA Group. Được thực hiện trên 2 loại phần mềm Cost Of The Shelf software và Custom Software.

- Với COTS Software, sẽ có 2 giai đoạn là alpha và beta. nhiều khách hàng sẽ được chọn để thử nghiệm và gửi feedback lại.

- Với Custom Software, SQA Group phải bảo đảm rằng Acceptance Test (bảo đảm đúng theo đặc tả) phải hiệu quả. Nếu không developer sẽ bị mất uy tín.

- Kiểm tra bao gồm các phương pháp:

- + Black Box
- + Robustness of Software.
- + Phải thỏa mãn tất cả các ràng buộc của phần mềm.
- + Kiểm tra toàn bộ code + tài liệu gửi đến cho khách hàng.

40. Kiểm thử chấp nhận là gì? Trong đó có những kiểm thử nào được thực hiện? Phân biệt.

- (Acceptance Test) là kiểm tra xem phần mềm đã hoạt động đúng như đặc tả đưa ra bởi developer hay chưa. Acceptance Test được thực hiện trên dữ liệu thực tế. Được thực hiện bởi 1 trong 3 nhóm đối tượng: đại diện khách hàng và nhóm SQA, hoặc nhóm SQA do khách hàng thuê.
- Có các kiểu kiểm thử sau:
 - + Kiểm thử đúng (Correctness testing)
 - + Hiệu năng (Performance) (tùy chọn)
 - + Robustness
 - + Tài liệu (Documentation)

41. Mô hình CMM là gì? Có những mức tăng trưởng nào trong mô hình CMM? Nội dung của mỗi mức?

- (capability maturity models) : Là các chiến lược có thể sử dụng để cải thiện quy trình phần mềm mà không phụ thuộc vào bất cứ một mô hình vòng đời phần mềm (life-cycle model) nào cả.
- CMM được chia thành các loại sau:
 - +CMMs for software (SW-CMM)
 - +Management of human resources (P-CMM; the P stands for “people”)
 - +Systems engineering (SE-CMM)
 - +Integrated product development (IPD-CMM)
 - +Software acquisition (SA-CMM).
- Các mức tăng trưởng trong mô hình CMM: 5 mức
 - + Lv1 : Initial Level: không có công nghệ phần mềm được áp dụng ở đây. Mọi thứ đều thực hiện 1 cách rất cơ bản. Tuy nhiên do thiếu kinh nghiệm nên nhìn chung giá thành và thời gian sẽ bị đội lên.
 - + Lv2: Repeatable Level: Ứng dụng những kỹ năng công nghệ phần mềm cơ bản vào công việc. Có sự quản lý về thời gian và giá thành cho dự án. Khi có sự cố nhờ có sự tính toán nên có thể kiểm soát được tình hình.
 - + Lv3: Defined Level: Quy trình phần mềm đã có đầy đủ tài liệu. Kỹ thuật quản lý cũng như hỗ trợ kỹ thuật được định nghĩa rõ ràng. Và có sự cố gắng tối đa để cải thiện quy trình dự án. Trong level này, giới thiệu công nghệ mới chỉ làm cho công việc bị gián đoạn và k có lợi.
 - + Lv4: (Managed Level) Từ mức này rất ít công ty có thể đạt đến. Đặt ra chất lượng và năng suất mục tiêu cho từng dự án. 2 yếu tố trên được xác định thường xuyên và thực hiện các biện pháp cần thiết để bảo đảm mục tiêu. Sử dụng quản lý chất lượng thống kê (Statistic Quality Control).

+ Lv5: (Optimizing Level): Tiếp tục cải thiện quy trình phần mềm, kỹ thuật quản lý quy trình và chất lượng thống kê (Statistical Quality & Process Control Techniques) được ứng dụng để định hướng tổ chức. Kinh nghiệm cho dự án trước phục vụ cho dự án sau. Và có sự tiếp thu phản hồi ý kiến khách hàng, cải thiện năng suất & chất lượng.

42. Làm thế nào để một tổ chức đạt được các mức tăng trưởng trong CMM?

Đây là giải pháp và thước đo về các mức tăng trưởng?

- Lv1: Không cần làm gì cũng được rồi.

- Lv2:

- + Quản lý lấy yêu cầu
- + Kế hoạch cho dự án phần mềm
- + Theo dõi và có tầm nhìn xa với dự án phần mềm.
- + Quản lý hợp đồng phần mềm.
- + SQA - bảo đảm chất lượng phần mềm.
- + Software Configuration Management.

- Lv3:

- + Tập trung vào quy trình của tổ chức
- + Định nghĩa các quy trình của tổ chức.
- + Chương trình huấn luyện
- + Quản lý phần mềm tích hợp.
- + Công nghệ dự án phần mềm.
- + Phối hợp liên nhóm
- + Peer Reviews.

- Lv4:

- + Quản lý định lượng quy trình.
- + Quản lý chất lượng phần mềm.

- Lv5:

- + Ngăn chặn lỗi.
- + Quản lý thay đổi công nghệ
- + Quản lý thay đổi quy trình

